



Lecture 5: (Scripting), Turtle Graphics, Pygame and Game development

Instructor: Jordan Schwartz

Slides adapted from Tony Liu

Logistics

- HW1 due this morning
 - Submitted on gradescope
- HW2 due next wednesday

Week 4

- Quick recap
- Scripting (from last week)
- Overview of Recursion
- Turtle Graphics
- Pygame
- Python game design principles



Recap

Exceptions

- Errors detected during program execution
 - Different from syntax errors, which are detected by the interpreter before runtime
- Examples

```
>>> next(it)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

```
>>> 1/0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

```
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

Handling Exceptions

- We handle exceptions with `try except` blocks
 - Just like Java's try-catch blocks

Modules

- Writing a `.py` file creates a module - this saves our function and class definitions
- Can be imported into other modules and can import other modules
- Modules are saved by their file name minus the `.py` and can be accessed by `__name__`



Scripting

HTTP requests

- HTTP (hypertext transfer protocol) is how client computers and servers exchange information over the internet
 - see the shared 19xx lecture on Internet Basics for more details
- HTTP
- the client (usually your computer's web browser) sends a HTTP request to a specific URL, and the server responds with data, such as the HTML for a webpage
- we can also send and receive requests programmatically in our Python code

Example: Wikipedia Summaries

- wikipedia has an API to extract article summaries
 - https://en.wikipedia.org/api/rest_v1/page/summary/article_name
- sending HTTP GET requests will return summary data about the article

Parsing data: JSON

- data from the internet needs to follow particular standards in order to be parsed by machines/code
- JSON is a particularly popular data standard to do this → hierarchical data
- JSON concepts map cleanly onto Python
 - JSON objects `{}` → dictionaries
 - JSON arrays `[]` → lists
 - JSON values `"", 1, 1.0, true` → strings, ints, floats, bools
- use the JSON Python standard library for parsing

requests: high-level HTTP API

- requests is a 3rd party module that abstracts HTTP response/requests into Python objects
- for reference, compare to lower-level urllib module in the Python standard library

Installing 3rd party modules

- we use pip to install third party modules

`pip3 install module` or `python3 -m pip install module`



Overview of Recursion

Recursion

- A recursive function is a function that executes itself in its own code block body
- Consists of:
 - Base case
 - Recursive call(s) (if multiple recursive calls made in one call to the outer function → tree recursive)
 - Recursive calls “break down” the problem in some way, like adding or subtracting to a parameter
 - Building off the recursive call results (recursive leap of faith)

[code demo]



Turtle Graphics

Turtle Graphics

- Is a module in Python
- A turtle, similar to a sprite, is a little robot
- `import turtle`

[code demo]

Check in C12

A lot of learning to use modules is learning how to read documentation. There are tons of functions available to you in turtle graphics we haven't gone over in class (yet).

With the person next to you, ONE person should pull up this page of the turtle documentation to explore! Find at least one turtle method you think is interesting, and write a short(!) program using it! (<https://docs.python.org/3/library/turtle.html> or google turtle graphics documentation)

We will test out 1-2 of your programs together.



Game Development

7 Stages of Game Development*

1. Planning
2. Pre-production
3. Production
4. Testing
5. Pre-launch
6. Launch
7. Post-launch

*this is from <https://gamemaker.io/en/blog/stages-of-game-development>, which I personally like to follow, but not all game developers will follow these exact steps

7 Stages of Game Development

1. Planning

- Genre?
- 2D or 3D?
- Art style?
- Gameplay mechanics?
- Hero vs. villain?
- Game engine?

Proof of concept:

- Determine what resources you'll need

2. Pre-production

- Scope out the work
- Create storyboards
- Create prototypes
- Decide on ideas, scrap others

3. Production

- Actually do the coding, creation, art/sound, etc.

7 Stages of Game Development

4. Testing

- In reality, should be testing as you go
- Check for bugs, glitches, exploits, or softlocks

6. Launch

- Squash remaining bugs,
- Implement quality of life improvements
- Add final artistic touches

5. Pre-launch

- Marketing
- Stable beta ready for the public
- Make trailers demos, early access release

7. Post-launch

- The game is out in the world!
Rake in the rewards
- Keep up with bug fixes
- DLC
- Free updates
- Balancing patches

Discuss with your table

What is the most important game development step? Why?

Briefly summarize your group's conclusion (or disagreement) in **L11**

Preview: HW2

You'll be doing steps 1 (planning), 2 (pre-production), 3 (production), and lightly 4 (testing) using the Pygame module in Python:

You will design, prototype, code, and debug 2 game mechanics for an existing simple base game!

Pygame

- Recap on imports modules
 - `import random`
- What is Pygame, how does it work?
 - Connections to OOP
- Initialization steps
 - Global vars vs instance and attribute vars
- `pygame.Rect`, (Vs like adding an image)
 - `colliderect`
- `pygame.image.load(relative_filepath).convert_alpha()`
- `font.render`
- `keys = pygame.key.get_pressed()`
- While loop that allows for animation
 - `CLOCK.tick`
 - `pygame.display.flip()`
- `SCREEN.blit` (blit in general)
- `pygame.quit()`

What is Pygame?

- A Python library for making 2D games
- Handles:
 - Window creation
 - Drawing graphics
 - Keyboard input
 - Collision detection
 - Timing
 - Fonts & text
 - Images

The way we're gonna teach Pygame...

... is by walking through the components of the base game of your homework.

Is it comprehensive of all of Pygame?

- NO!

Is it enough to complete your homework assignment?

- Probably not...?

Will this assignment require you to practice reading documentation?

- Hopefully!

- But you can always come to OH and Ed for help!

Feel free to download and open up the HW file to follow along as long as you can 1) keep paying attention with an open computer and 2) will find it productive in order to take notes or write comments in the file

Quick overview of the homework base game

- The player moves a character around a bounded arena using the arrow keys
 - cannot pass through walls.
- Random food items spawn inside the play area, and the player earns points by collecting them.
- The game continuously updates movement, collision detection, and score display in a real-time game loop running at 60 FPS.

How is Pygame Structured?

1. Imports
2. Initialization
3. Constants
4. Game objects
5. Game state variables
6. Main game loop
 - a. Event handling
 - b. Update logic
 - c. Collision handling
 - d. Drawing
7. Quit

To start we need to import modules!

- You might need to start by installing pygame with `pip install pygame` or `pip3 install pygame`
- The 2 imports we provide you with are for `pygame` and for `random`

```
HW2 > 🐍 HW2_base_game.py > ...  
1     import pygame  
2     import random  
3
```

- You can always add more as needed!

Quick aside on the Random module

- Pseudo-random
- Uses a seed, defaults to the system's time
- (side note: it's not strong enough for most security needs, there's another module for that)

[code demo]

- In this example, we will need random for randomly spawning food!

Initialization

- `pygame.init()`
 - Initializes all pygame modules (display, font, sound, etc.)
 - Placed before using pygame features

```
4      # -----  
5      # Initialization  
6      # -----  
7      pygame.init()  
8
```


Creating the Game Window

- Sets the resolution
- Creates the SCREEN object, which is the “surface” of the game
 - Everything drawn, all images made go onto the screen surface
- Also sets the window caption

```
9  WIDTH, HEIGHT = 600, 600
10 SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
11 pygame.display.set_caption("Simple Food Collector")
12
```

Game Clock (Frame Rate Control)

- Sets the frames per second (FPS)
- If you don't limit the FPS, game loops run as fast as possible
 - so this allows you to control how often images are refreshed

At the beginning in init phase

```
13  CLOCK = pygame.time.Clock()
```

Inside the
main game loop

```
93  while running:  
94      |  CLOCK.tick(60)
```

Fonts and Text

```
14  FONT = pygame.font.SysFont(None, 36)
```

Create a font object

```
164  score_text = FONT.render(f"Score: {score}", True, BLACK)
165  SCREEN.blit(score_text, (WALL_THICKNESS + 10, 5))
```

- Use that font object to render text (notice the formatting with f-strings!)
 - Render: create a Surface object, like the SCREEN, memory of the pixel data
- Then blit that object onto the existing SCREEN object

```
167  pygame.display.flip()
```

- Flip actually updates the window at the end of the game loop

Game Constants

- They exist
- Consistency
- Readability

```
25  # -----
26  # Game Constants
27  # -----
28  WALL_THICKNESS = 20
29  PLAYER_SIZE = 30
30  FOOD_SIZE = 20
31  PLAYER_SPEED = 5
32  POINTS_PER_FOOD = 10
33
```

Rectangles

`pygame.Rect()`

- In this base game they:
 - Represent player
 - Represent walls
 - Represent food
 - Represent background bounds

- Settings of a rect
 - `.x`, `.y`
 - `.left`, `.right`, `.top`, `.bottom`
 - `.colliderect()`
- Check out the...
 - Walls
 - Player character
 - Food
 - etc.

```
pygame.Rect(40, 40, wall_dimensions[0][0], wall_dimensions[0][1])
```

```
68 def spawn_food():
69     return pygame.Rect(
70         random.randint(
71             WALL_THICKNESS + 40,
72             WIDTH - WALL_THICKNESS - FOOD_SIZE - 40
73         ),
74         random.randint(
75             WALL_THICKNESS + 40,
76             HEIGHT - WALL_THICKNESS - FOOD_SIZE - 40
77         ),
78         FOOD_SIZE,
79         FOOD_SIZE
80     )
```

Loading and Scaling Images

- Check out the docs for more info, but there's plenty of examples in the base game

```
151 # Draw player
152 player_image = pygame.image.load("assets/bread.jpg").convert_alpha()
153 player_scaled = pygame.transform.scale(player_image, (PLAYER_SIZE, PLAYER_SIZE))
154 SCREEN.blit(player_scaled, player)
155
```

Game State Variables

- Variables for which the game state, such as if it should still be running, are defined
- `running` controls loop lifetime
- `score` tracks progress

```
84  # -----  
85  # Game State  
86  # -----  
87  score = 0  
88  running = True
```

Event Queue

```
for event in pygame.event.get():  
    if event.type == pygame.QUIT:  
        running = False
```

- list of things that have happened since the last frame.
- Examples of events:
 - A key was pressed or released
 - The mouse moved or clicked
 - The user clicked the window's X button
- Pygame automatically collects these and stores them in a queue.
- You access them with: `pygame.event.get()`
 - Retrieves all pending events
 - Removes them from the queue
 - Lets you respond to them

Keyboard input

```
keys = pygame.key.get_pressed()
```

Speed

Speed determines the distance a component should travel each game loop. `dx` sets the horizontal speed and `dy` sets the vertical

```
# -----  
# Movement  
# -----  
keys = pygame.key.get_pressed()  
dx = dy = 0  
  
if keys[pygame.K_LEFT]:  
    dx = -PLAYER_SPEED  
if keys[pygame.K_RIGHT]:  
    dx = PLAYER_SPEED  
if keys[pygame.K_UP]:  
    dy = -PLAYER_SPEED  
if keys[pygame.K_DOWN]:  
    dy = PLAYER_SPEED
```

Movement and collision

Then we actually move the player by adjusting their rect x and y settings which updates in memory

We use the `rectobj.colliderect(otherrectobj)` to detect if things have collided, rather than calculating it ourselves

```
# Move player and handle wall collision
player.x += dx
for wall in walls:
    if player.colliderect(wall):
        if dx > 0:
            player.right = wall.left
        if dx < 0:
            player.left = wall.right

player.y += dy
for wall in walls:
    if player.colliderect(wall):
        if dy > 0:
            player.bottom = wall.top
        if dy < 0:
            player.top = wall.bottom
```

Try playing the game!

Try playing the base game now, start brainstorming what your next steps might be to come up with more game mechanics

Thanks !

Do you have any questions?

youremail@freepik.com

+34 654 321 432

yourwebsite.com



CREDITS: This presentation template was created by **Slidesgo**, and includes icons, infographics & images by **Freepik**

Please keep this slide for attribution

Instructions for use

If you have a free account, in order to use this template, you must credit [Slidesgo](#) by keeping the [Thanks](#) slide. Please refer to the next slide to read the instructions for premium users.

As a Free user, you are allowed to:

- Modify this template.
- Use it for both personal and commercial projects.

You are not allowed to:

- Sublicense, sell or rent any of Slidesgo Content (or a modified version of Slidesgo Content).
- Distribute Slidesgo Content unless it has been expressly authorized by Slidesgo.
- Include Slidesgo Content in an online or offline database or file.
- Offer Slidesgo templates (or modified versions of Slidesgo templates) for download.
- Acquire the copyright of Slidesgo Content.

For more information about editing slides, please read our FAQs or visit our blog:

<https://slidesgo.com/faqs> and <https://slidesgo.com/slidesgo-school>